Bangladesh Army University of Science and Technology, Khulna
Department of Computer Science and Engineering

PROJECT REPORT

CakeCraft

A Web-Based Custom Cake Ordering, Pricing and Delivery Tracking System

Submitted By:
Name: MD. MANAM SHAFIL KHAN
UID: 1072410005101014
Department: Computer Science and Engineering
Level: 03 Term: 02

Submitted To:
Name: Sumya Akter
Lecturer, Department of CSE
BAUST, Khulna

Course Title: Software Development Project-I
Course Code: CSE-364
Submission Date: 06 November 2026

Project Repository: the complete source code and assets of this project are showcased on GitHub (Link: https://github.com/Md-Manam-Khan/CakeCraft)

Acknowledgement

I would like to express my sincere gratitude to my course instructor, **Sumya Akter**, Lecturer, Department of Computer Science and Engineering, Bangladesh Army University of Science and Technology (BAUST), Khulna, for her guidance, encouragement and support throughout the Software Development Project-I course. Her feedback helped me shape CakeCraft from a simple sign-up and sign-in page into a complete ordering and delivery platform.

I am also thankful to the Department of Computer Science and Engineering for providing the learning environment, and to my friends and family for their constant motivation.

The entire code of CakeCraft was written by me. I also acknowledge the AI tools used along the way: the background images used across the site were generated with **ChatGPT**, and **Claude** (an AI assistant by Anthropic) found security loopholes and bugs in my code, which I then fixed myself to make the system better.

Abstract

This project, titled **CakeCraft**, aims to solve the problem of ordering custom cakes in a slow and error-prone manner, where customers describe their requirements over phone calls or chat messages, the price is negotiated by hand, and nobody can easily see where the order is until it arrives. CakeCraft replaces this with a single web platform on which a customer designs a cake, sees the price update live and tracks the order until it reaches the door.

The system is developed using **HTML5** and **CSS3** on the front end, **PHP** (procedural, mysqli with prepared statements and sessions) on the back end, and a **MySQL** relational database, running on an **Apache** server through XAMPP. It provides features such as secure customer registration and login with hashed passwords, protection against forged requests (CSRF tokens) and repeated password guessing (login-attempt limiting), a cake customization form with dynamic pricing, customer order history with live status and rider updates, an admin panel for stock, menu, prices, orders, rider management and the admin's own credentials, an order-assignment workflow, and a rider panel to accept or decline deliveries and post live updates.

The source code and other assets of the project are publicly available on the author's GitHub page: https://github.com/Md-Manam-Khan/CakeCraft

Table of Contents

Acknowledgement 2

Abstract 2

Table of Contents 3

1. Introduction 4

1.1 Project Repository 4

2. Objectives 4

3. Project Requirements 5

3.1 Functional Requirements 5

3.2 Non-Functional Requirements 5

3.3 Software Requirements 6

3.4 Hardware Requirements 6

4. System Design 6

4.1 System Architecture 6

4.2 Use Case Diagram 7

4.3 Order Flowchart 8

4.4 Database Design 9

5. Tools and Technologies 10

6. Implementation 11

6.1 User Interface 11

6.2 Login and Registration 12

6.3 Main Functionalities 13

6.4 Database Connectivity 17

6.5 CRUD Operations 18

7. Testing and Results 18

8. Conclusion 19

Possible Future Improvements 19

9. References 20

1. Introduction

Cakes are an important part of birthdays, weddings, anniversaries and everyday celebrations. A custom cake, however, involves several decisions at once: the flavor, the filling, the size, a message on top, allergy notes and a delivery address. In most small bakeries these details are still collected informally through phone calls or social-media messages. This leads to misunderstandings, inconsistent prices, forgotten requirements and no reliable way for a customer to know the progress of an order.

CakeCraft is a full-stack web application that solves this problem. A customer can register, design a cake (flavor, filling, size, delivery address and special instructions) and place the order while watching the total price change instantly according to prices set by the shop owner. The shop owner uses a dedicated admin panel to manage stock, edit the menu and prices, review incoming orders, create rider accounts, assign each accepted order to a delivery rider and change the admin's own login credentials. The rider uses a separate panel to accept or decline the assignment and to post updates such as *Received*, *Reached* or *Delivery done*, which the customer can read under My Orders.

Because the system stores personal data and handles orders and prices, it also includes security measures: hashed passwords, CSRF tokens on every form, limits on repeated failed logins and an automatic logout after a period of inactivity.

The project was developed as part of the course CSE-364 Software Development Project-I. It started as a basic authentication system and was extended step by step into a working ordering, pricing, tracking and delivery platform. All prices are shown in Bangladeshi Taka (for example 850 TK).

1.1 Project Repository

The complete source code, database script and image assets of CakeCraft are showcased on my GitHub page, where they can be viewed, downloaded and run locally:
https://github.com/Md-Manam-Khan/CakeCraft

2. Objectives

The main objective of the project is to build a user-friendly, database-driven web application for custom cake ordering and delivery. The specific objectives are:

- To develop a user-friendly application in which customers can design a cake and place an order without any manual communication.
- To implement the main project features: registration and login, cake customization, dynamic pricing, order tracking, admin management and rider delivery updates.
- To manage data efficiently in a relational MySQL database with proper keys and relationships between users, orders, riders and delivery updates.
- To give each type of user (customer, admin and rider) a separate, session-protected interface that shows only what that user needs.
- To protect user data and the system by storing passwords as hashes, using prepared statements for all database queries, validating input on the server, protecting forms with CSRF tokens and limiting repeated failed logins.
- To keep the design responsive and visually appealing, so that the system is comfortable to use on both desktop and mobile screens.

3. Project Requirements

3.1 Functional Requirements

Customer

- Register with full name, email, phone number, a unique ID name and a password of at least 8 characters.
- Sign in using the ID name, email address or phone number together with the password.
- Choose a cake flavor, filling and size from the menu, enter a delivery address and optional special instructions.
- See the total price update live while choosing, and place the order.
- View all past and current orders with their status (pending, accepted, declined, delivered) and the rider's updates with time stamps.

Admin

- Log in through a separate admin login page with an admin account stored in the database.
- Change the admin profile (username) and password from the Edit Credentials popup.
- Add ingredients to stock or top up the quantity of an existing ingredient.
- Add or remove cake flavors and fillings, and edit the price of any flavor, filling or size.
- Review pending orders and accept or decline them.
- Create rider accounts and assign an accepted order to a chosen rider; re-assign it if the rider declines.
- See the assignment state of every order (awaiting rider, accepted, declined, delivered).

Rider

- Log in with the credentials created by the admin.
- See only the orders assigned to him or her and accept or decline each one.
- For an accepted order, view the customer, address and total, send progress updates and press Delivery Done.

3.2 Non-Functional Requirements

- **Security:** hashed passwords, prepared statements, server-side validation of every input, CSRF tokens on every form, separate session variables for each role, a new session id after login, automatic logout after 30 minutes of inactivity and temporary blocking after repeated failed logins.
- **Usability:** simple forms, popup boxes for menu and credential editing and clear status badges.
- **Responsiveness:** the layout adapts to smaller screens through CSS media queries.
- **Consistency:** the price saved with an order is calculated on the server from the database prices at the moment the order is placed.

3.3 Software Requirements

Item	Details
Programming languages	PHP (back end), HTML5 and CSS3 (front end), SQL
Database	MySQL (managed through phpMyAdmin), InnoDB engine, utf8mb4 character set
Web server	Apache HTTP Server, provided by XAMPP
Version control / hosting	Git and GitHub
Client	Any modern web browser (Chrome, Edge, Firefox)
Operating system	Windows (development), any OS that can run XAMPP

3.4 Hardware Requirements

A standard personal computer with at least 4 GB of RAM and a dual-core processor is sufficient to run the XAMPP stack and the application locally.

4. System Design

4.1 System Architecture

CakeCraft follows a classic three-tier web architecture. The browser (customer, admin or rider) sends HTTP requests to PHP scripts running on the Apache server. The scripts validate the session, check the CSRF token of every submitted form, run prepared SQL statements against the MySQL database and return HTML pages to the browser. Each role has its own session variable (`user_id`, `admin_logged_in`, `rider_id`), so a page of one role cannot be opened by another.

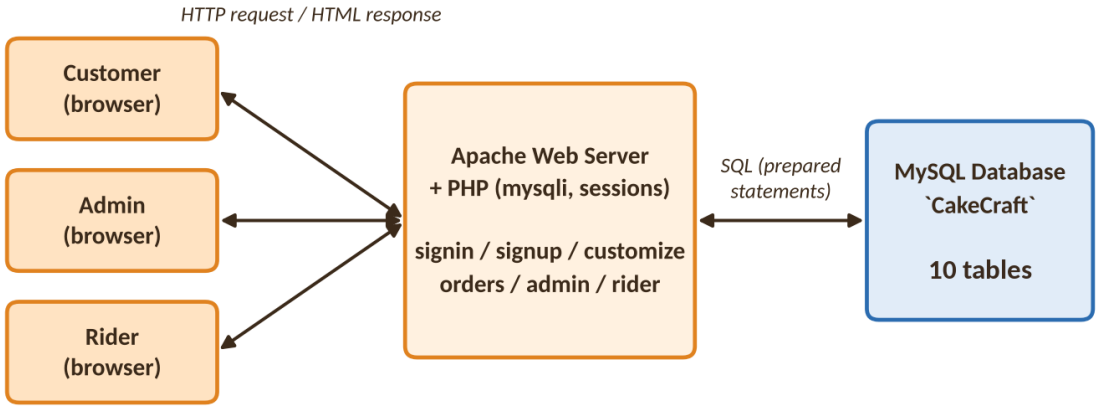


Figure 4.1: Three-tier architecture of CakeCraft

4.2 Use Case Diagram

The system has three actors. The customer designs and orders cakes and tracks them; the admin manages the shop, edits the admin credentials and assigns deliveries; the rider performs the delivery and posts updates.

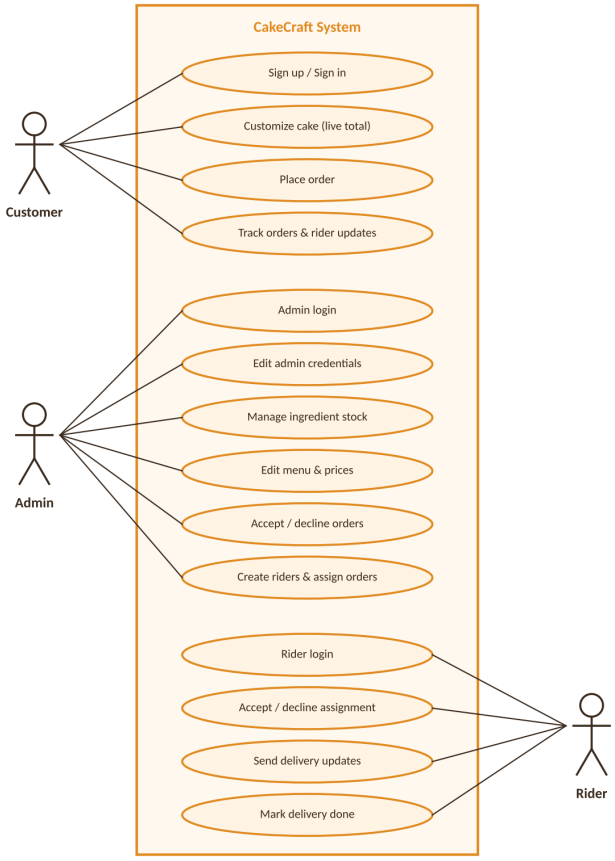


Figure 4.2: Use case diagram of CakeCraft

4.3 Order Flowchart

The life of an order is controlled by two status columns. `orders.status` moves from *pending* to *accepted* or *declined* (admin) and finally to *delivered* (rider). `orders.rider_status` is empty until the order is assigned, becomes *pending* when the admin assigns a rider, and then *accepted* or *declined* according to the rider's answer. A declined assignment returns to the admin for re-assignment.

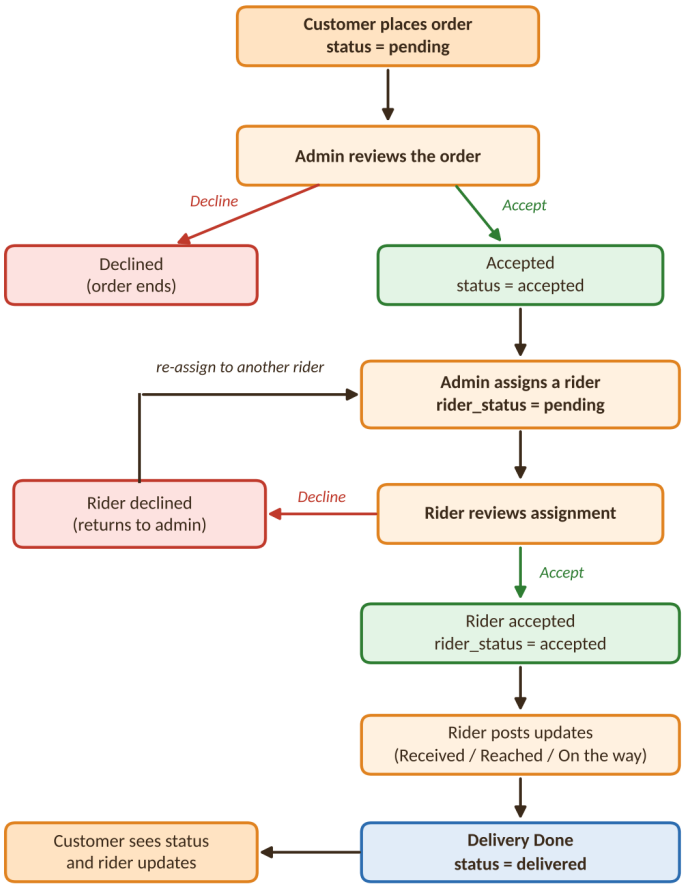
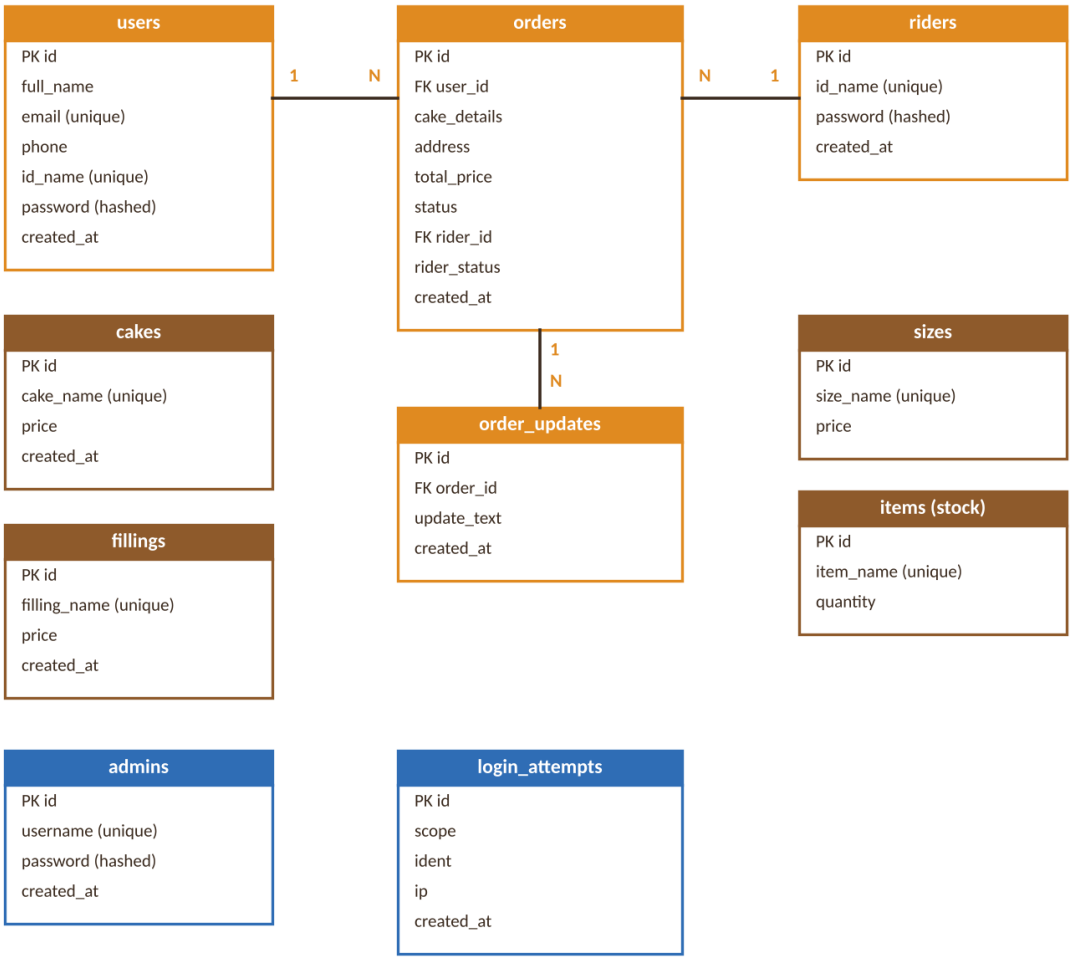


Figure 4.3: Order lifecycle from placement to delivery

4.4 Database Design

The database `CakeCraft` contains ten tables. *users*, *riders*, *orders* and *order_updates* are linked with foreign keys; *cakes*, *fillings* and *sizes* hold the menu with a price for each entry, *items* holds the ingredient stock, *admins* holds the admin accounts with hashed passwords, and *login_attempts* records recent failed logins so that repeated password guessing can be blocked.



Brown tables: menu / stock data (prices are looked up by name when an order is placed).

Blue tables: security data (admin accounts are stored in admins; login_attempts records failed logins).

Figure 4.4: Entity-relationship diagram of the CakeCraft database

Table	Purpose	Key columns
users	Customer accounts	id, full_name, email (unique), phone, id_name (unique), password (hash), created_at
riders	Delivery accounts created by the admin	id, id_name (unique), password (hash), created_at
admins	Admin accounts (the admin can edit them)	id, username (unique), password (hash), created_at
login_attempts	Failed logins, used to block repeated guessing	id, scope, ident, ip, created_at

Table	Purpose	Key columns
items	Ingredient stock	id, item_name (unique), quantity
cakes	Cake flavors on the menu	id, cake_name (unique), price, created_at
fillings	Fillings on the menu	id, filling_name (unique), price, created_at
sizes	Cake sizes	id, size_name (unique), price
orders	Customer orders	id, user_id (FK), cake_details, address, total_price, status, rider_id (FK), rider_status, created_at
order_updates	Rider messages shown to the customer	id, order_id (FK), update_text, created_at

The script `database.sql` creates all tables and seeds the menu with five starter cakes (Chocolate Truffle, Vanilla Bean, Red Velvet, Black Forest and Strawberry Shortcake), three fillings (Vanilla, Chocolate and Strawberry) and three sizes (Small, Medium and Large), each with a starting price. It also creates one default admin account whose password is stored only as a hash; the admin is expected to replace it through Edit Credentials.

5. Tools and Technologies

Layer / purpose	Technology	Use in CakeCraft
Front end	HTML5, CSS3	Page structure and styling; responsive layout with media queries, popup (overlay) boxes, 3D-press buttons and themed backgrounds in a single shared stylesheet `style.css`
Interactive features	Browser interface	Live total price on the customization page; popup boxes and dropdown menu in the admin panel (menu editing and Edit Credentials); rider selection for assignment
Back end	PHP (procedural)	Business logic, form handling, sessions, role-based page protection, CSRF tokens, login-attempt limiting, password hashing with `password_hash()` and `password_verify()`
Database access	MySQLi, prepared statements	Secure parameterised queries for all user input
Database	MySQL (InnoDB)	Ten relational tables with primary keys, unique constraints and foreign keys
Server	Apache via XAMPP	Local web server and MySQL service for development and testing
Database tool	phpMyAdmin	Creating the schema, importing `database.sql` and inspecting data
Version control	Git and GitHub	Source code hosting and showcase of the project
AI assistance	ChatGPT, Claude	ChatGPT generated the background images used on the site; Claude found security loopholes and bugs in the code, which I fixed myself

The project deliberately uses plain PHP, MySQL, HTML and CSS without any framework or library. This keeps the code easy to follow and makes every part of the request-handling, session and database logic visible, which suits the learning goals of a first software development project.

6. Implementation

The entire code of the project was written by me. It consists of 13 PHP pages, one HTML landing page, one shared stylesheet and one SQL script. The table below summarises the files.

File	Role
index.html	Landing page with Sign In, Sign Up, Admin Login and Rider Login buttons
config.php	Database connection, secure session start, security headers and helper functions (`tk()` price formatter, CSRF tokens, input validation, login-attempt limiting)
signup.php / signin.php / logout.php	Customer registration, login and logout
dashboard.php	Customer hub after login
customize.php	Cake order form with live total and server-side price calculation
orders.php	Customer order history, status and rider updates
admin_login.php / admin.php / admin_logout.php	Admin login (accounts in the admins table), admin panel with Edit Credentials, and logout
rider_login.php / rider.php / rider_logout.php	Rider login, rider panel and logout
style.css	Shared stylesheet for all pages
database.sql	Full schema (10 tables) and starter menu data

6.1 User Interface

The interface uses an orange-and-cream colour theme and a different cake photograph as background for each section of the site, which gives every role a recognisable identity. Forms are placed on a translucent white card so that they stay readable on top of the photographs. All the background images are AI-generated and were created with ChatGPT.

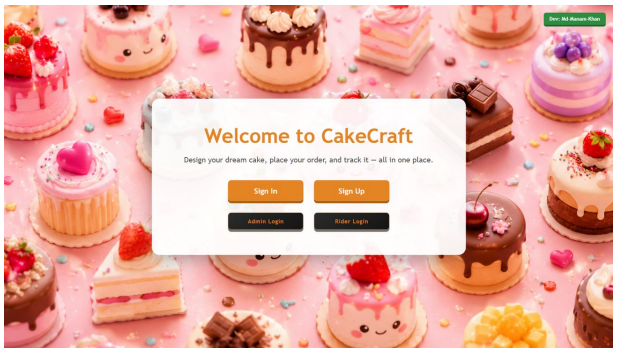


Figure 6.1: Landing page with customer, admin and rider entry points

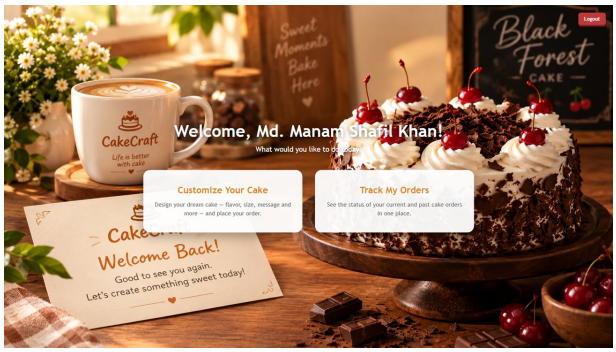


Figure 6.2: Customer dashboard after login

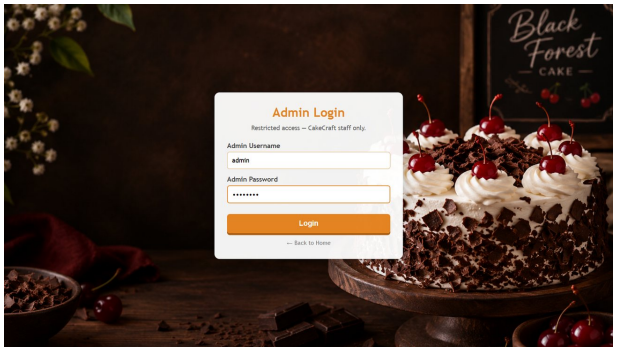


Figure 6.3: Admin login page

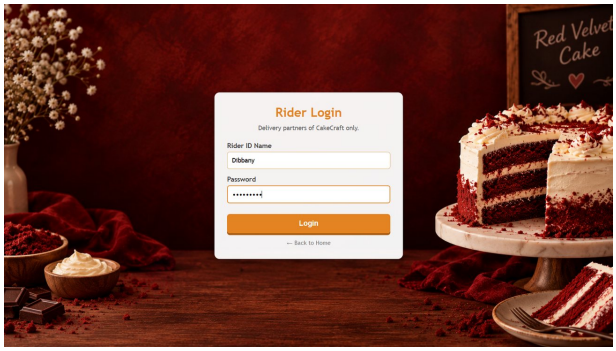


Figure 6.4: Rider login page

6.2 Login and Registration

Registration. A new customer fills in the sign-up form with full name, email, phone number, ID name and password. `signup.php` checks that no field is empty, that the email, phone number and ID name have a valid format, that the password is 8 to 72 characters long, and that the email and ID name are not already used. The password is never stored as plain text: it is converted with `password_hash()` and only the hash is inserted into the *users* table with a prepared statement. After a successful insert the user is logged in automatically and sent to the dashboard.

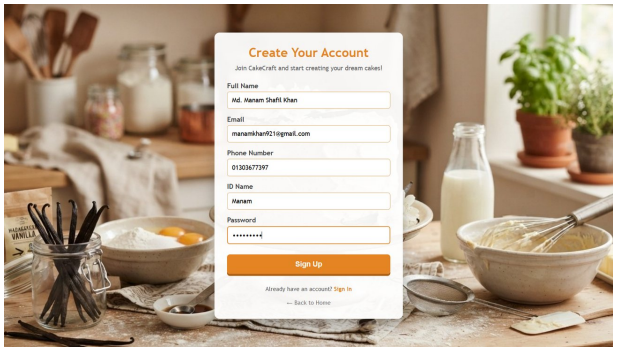


Figure 6.5: Customer registration form

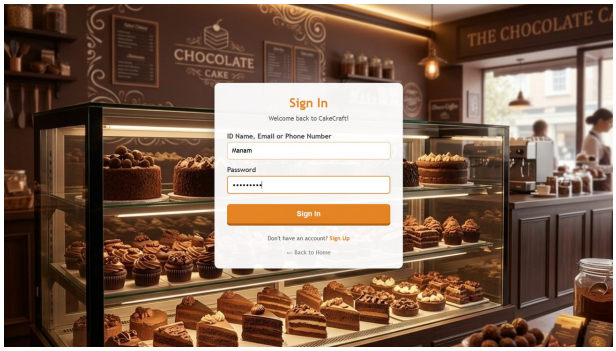


Figure 6.6: Customer sign-in (ID name, email or phone)

Listing 6.1 - Storing a hashed password (signup.php)

```
$hashpass = password_hash($password, PASSWORD_DEFAULT);
$stmt = $conn->prepare("INSERT INTO users (full_name, email, phone, id_name, password)
                        VALUES (?, ?, ?, ?, ?)");
$stmt->bind_param("sssss", $name, $email, $phone, $idname, $hashpass);
```

Sign in. The customer can log in with the ID name, the email address or the phone number. `signin.php` first checks whether too many failed attempts have been recorded, then looks the user up with one prepared query and verifies the password with `password_verify()`. On success the failed-attempt records are cleared, the session id is regenerated, the user's id and name are saved in the session and the dashboard opens. Pages such as `customize.php` and `orders.php` check the session first and redirect to the sign-in page if nobody is logged in.

Protection against guessing and forged requests. Every failed login is recorded in the *login_attempts* table. After 5 wrong attempts for the same ID, or 20 from the same IP address, within 15 minutes, further attempts are refused for a while; old records are deleted automatically. Every form carries a random CSRF token that is checked on the server before the request is processed, session cookies are marked HttpOnly and SameSite, and a user who stays idle for 30 minutes is logged out automatically. These protections were added after Claude, an AI assistant by Anthropic, found security loopholes and bugs in the first version of my code; I fixed them myself in the final update to make the system better.

Listing 6.2 - Verifying the password and starting the session (signin.php)

```
if (login_blocked($conn, 'user', $login)) {
    $msg = "Too many failed attempts. Please wait 15 minutes and try again.";
} else {
    $stmt = $conn->prepare("SELECT id, full_name, password FROM users
                            WHERE id_name = ? OR email = ? OR phone = ?");
    $stmt->bind_param("sss", $login, $login, $login);
    ...
    if (password_verify($password, $hash) && $user) {
        login_clear($conn, 'user', $login);
        session_regenerate_id(true);
        $_SESSION['user_id']   = $user['id'];
        $_SESSION['user_name'] = $user['full_name'];
        header("Location: dashboard.php");
    } else {
        login_fail($conn, 'user', $login);
    }
}
```

Admin and rider login. Admin and rider logins work the same way, including the attempt limit and the new session id, but set their own session variables. Riders are looked up in the *riders* table and admins in the *admins* table; both passwords are hashed. The admin account is no longer fixed in the source code: it lives in the database and can be changed by the admin through Edit Credentials (see Section 6.3.3).

6.3 Main Functionalities

6.3.1 Cake customization and dynamic pricing

The *Customize Your Cake* page reads the flavors, fillings and sizes from the database and shows the price beside every option. The *Total* box updates instantly whenever the customer changes a selection and adds up the prices of the chosen flavor, filling and size. For example, Chocolate Truffle (800 TK) + Vanilla filling (100 TK) + Medium size (600 TK) gives a total of 1500 TK.

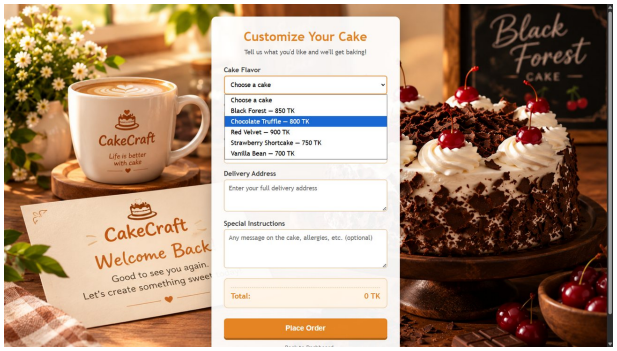


Figure 6.7: Flavor dropdown listing every cake with its price

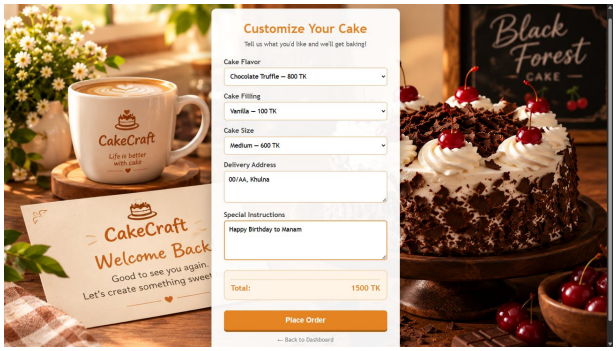


Figure 6.8: Completed form with the live total (1500 TK)

The total shown in the browser is only for display. When the order is submitted, `customize.php` looks up the three prices again in the database and calculates the total on the server, so the saved price always follows the prices set by the admin and cannot be changed from the browser. An item whose price is still 0 TK cannot be ordered. The order is stored with the status *pending*.

Listing 6.3 - Server-side price calculation and order insert (customize.php)

```
$total = $row1['price'] + $row2['price'] + $row3['price'];   // flavor + filling + size
$details = "Flavor: $flavor | Filling: $filling | Size: $size | Note: " . $notetxt;
$stmt = $conn->prepare("INSERT INTO orders (user_id, cake_details, address,
                        total_price, status) VALUES (?, ?, ?, ?, 'pending')");
```

6.3.2 Customer order tracking

My Orders lists every order of the logged-in customer, newest first, with the cake details, address, total, a coloured status badge and, once a rider is working on it, the rider updates with time stamps.

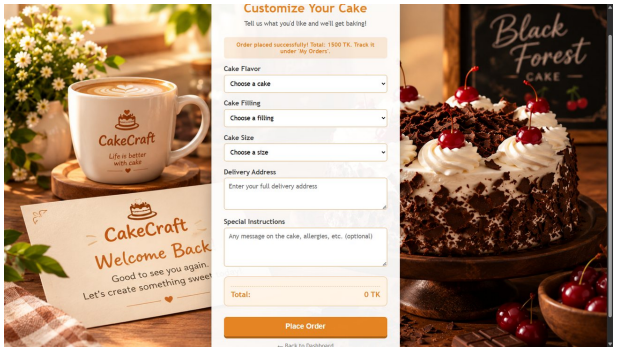


Figure 6.9: Confirmation message after placing an order

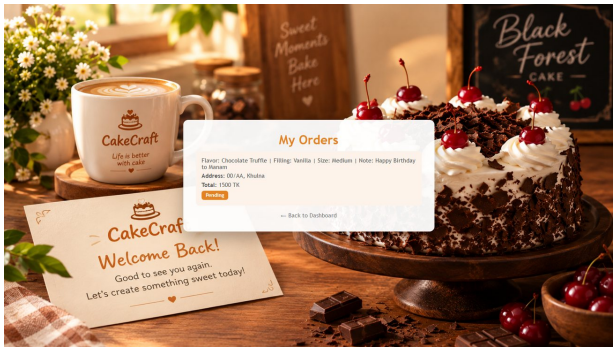


Figure 6.10: My Orders - the new order is Pending

6.3.3 Admin panel

The admin panel is a single page divided into boxes. The *Items* box adds a new ingredient or increases the quantity of an existing one. The *Current Orders* box shows every pending order as *Order #1*, *#2*, … with the customer, details, address and total, together with *Accept* and *Decline* buttons. The update query contains `AND status = 'pending'`, so an order that has already been handled cannot be changed again by mistake.

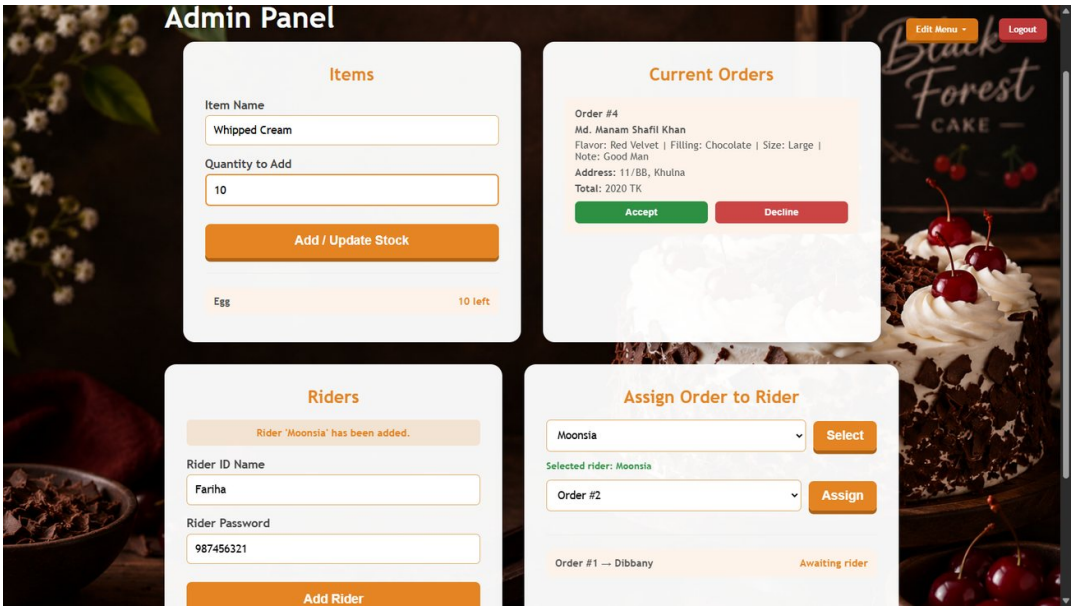


Figure 6.11: Admin panel - stock, pending order and rider creation

In the *Riders* box the admin creates a rider by typing an ID name and a password of at least 8 characters (stored hashed). In *Assign Order to Rider* the admin picks a rider, presses *Select*, chooses an accepted order by its number and presses *Assign*. A live list beneath shows every assignment and its state: awaiting rider, accepted, declined or delivered. An order whose rider declined, or has not yet answered, can be assigned to another rider.

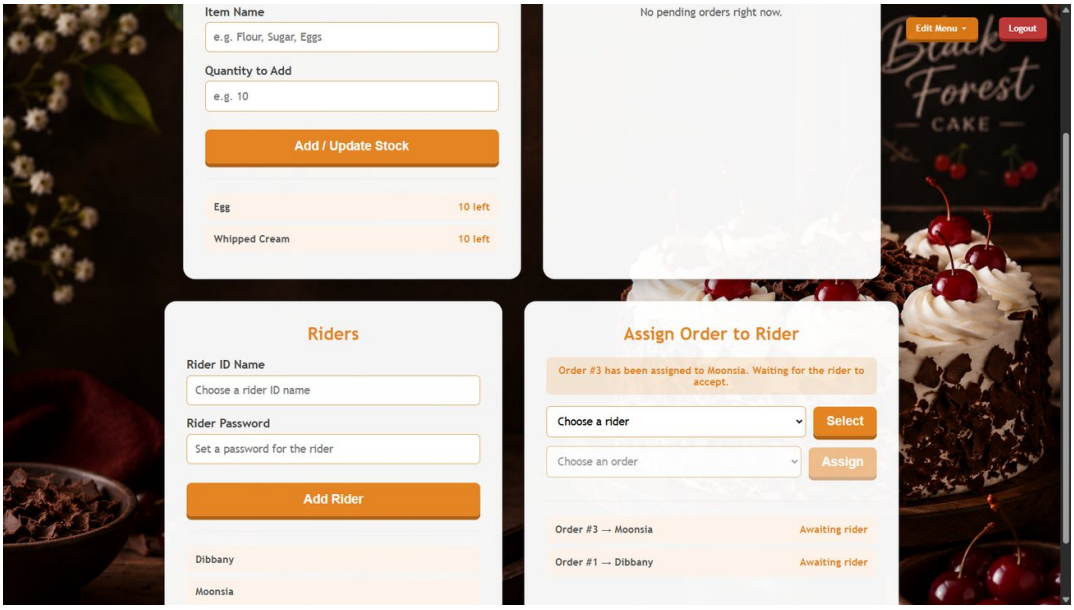


Figure 6.12: Assigning an accepted order to a rider

The *Edit Menu* dropdown in the top corner opens popup boxes to *Add Cake*, *Remove Cake*, *Add Filling*, *Remove Filling* and *Edit Price*. New items start at 0 TK until the admin sets their price. In *Edit Price* the admin chooses a flavor, filling or size from one grouped list, types the new price and saves; customers see the new price immediately. The table and column names used by this feature are chosen from a fixed list inside the script and never taken directly from user input.

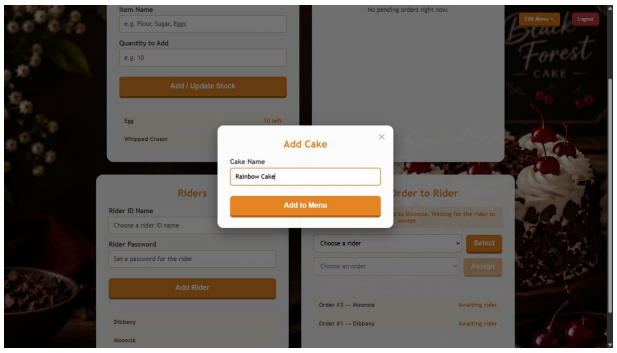


Figure 6.13: Add Cake popup

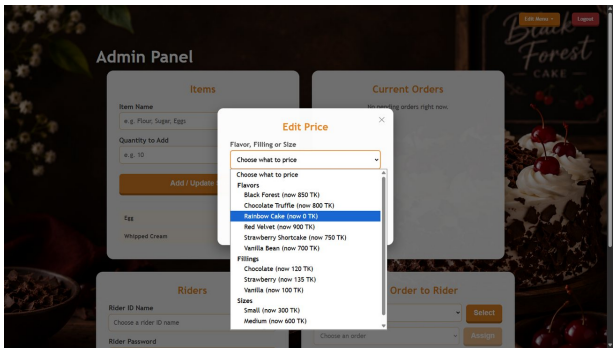


Figure 6.14: Edit Price - choosing a flavor, filling or size

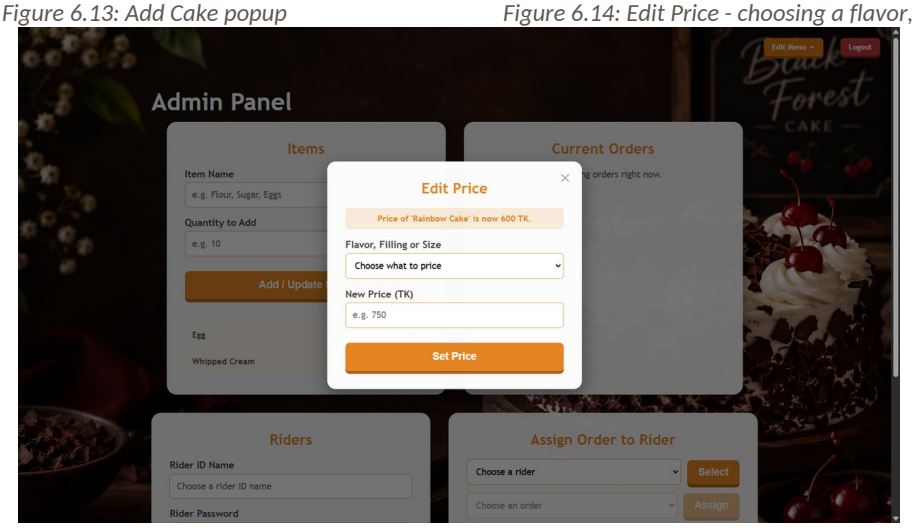


Figure 6.15: Confirmation after setting a new price

Edit Credentials. Next to the Edit Menu button, the *Edit Credentials* button opens a popup in which the admin types a new *Admin Profile* (username) and a new *Admin Password*, each twice for confirmation. The script checks that all four boxes are filled and that each pair matches, that the profile is 3 to 50 characters (letters, numbers, dot, underscore or dash), that the password is 8 to 72 characters and different from the

default one, and that no other admin already uses the profile. Only the hash of the new password is stored: the *admins* row is updated with a prepared statement and the session id is regenerated. This replaces the fixed admin credentials of the first version of the project.

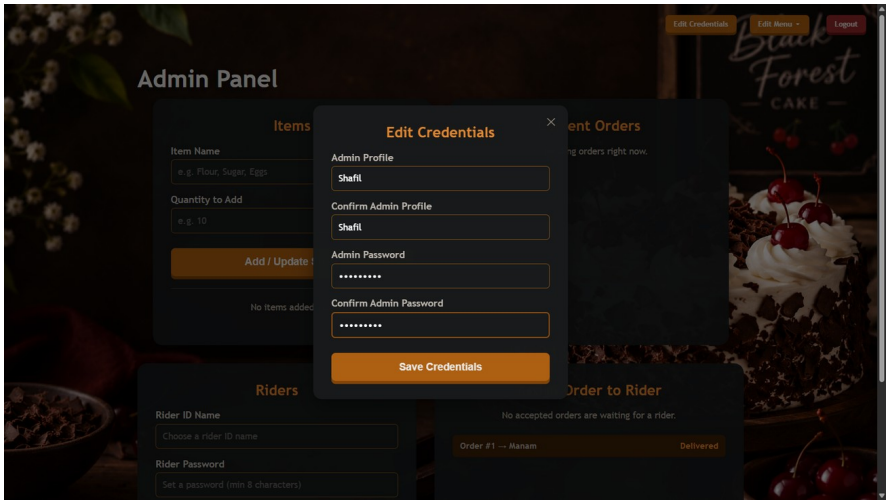


Figure 6.16: Edit Credentials popup - changing the admin profile and password

Listing 6.4 - Saving the new admin credentials (admin.php)

```
$newhash = password_hash($new_pass, PASSWORD_DEFAULT);
$update = $conn->prepare("UPDATE admins SET username = ?, password = ? WHERE id = ?");
$update->bind_param("ssi", $new_user, $newhash, $admin_id);
$update->execute();
session_regenerate_id(true);
$_SESSION['admin_name'] = $new_user;
```

6.3.4 Rider panel

After login the rider sees only the orders assigned to him or her. Under *New Assignments* the rider can *Accept* or *Decline*. A declined order disappears from the rider's panel and goes back to the admin. An accepted order moves to *Ongoing Orders*, which shows the customer, address and total, a text box for updates such as *Received*, *Reached* or *On the way*, and a *Delivery Done* button.

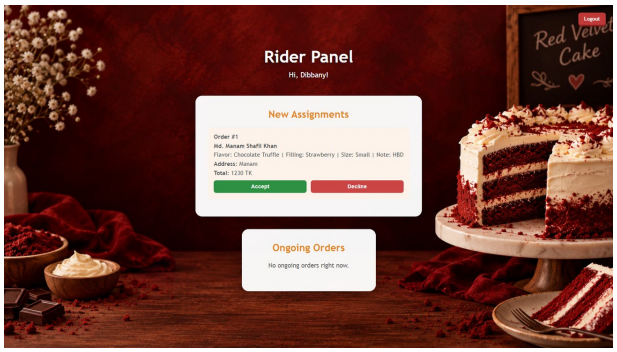


Figure 6.17: Rider panel - new assignment

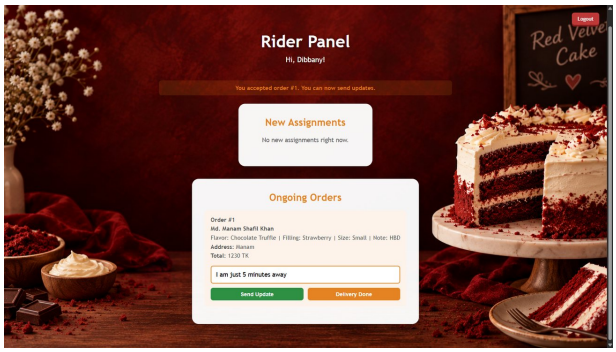


Figure 6.18: Rider panel - ongoing order with update box

When the rider presses *Delivery Done*, the script saves a *Delivery done* update and sets the order status to *delivered*. Both the updates and the final status are immediately visible to the customer.

Listing 6.5 - Completing a delivery (rider.php)

```
$insert = $conn->prepare("INSERT INTO order_updates (order_id, update_text)
                          VALUES (?, ?)");
...
$update = $conn->prepare("UPDATE orders SET status = ? WHERE id = ?");
// status = 'delivered'
```

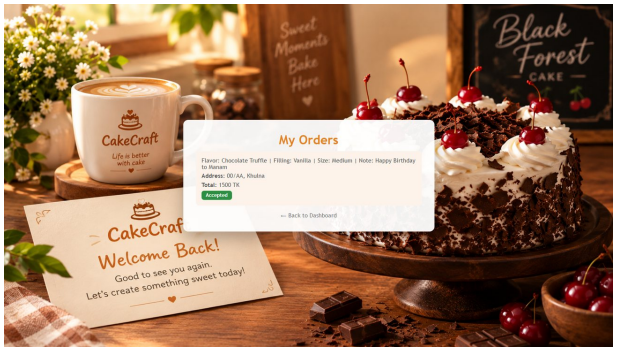


Figure 6.19: Customer view - order accepted by the admin

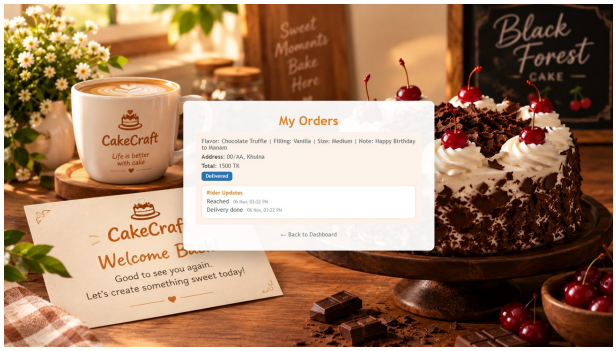


Figure 6.20: Customer view - delivered, with rider updates

6.4 Database Connectivity

All pages include `config.php`, which starts the session, applies the security settings and opens one MySQLi connection to the `CakeCraft` database. A small helper `tk()` formats prices as *850 TK*. Every query that contains user input uses a prepared statement with bound parameters, which protects the application against SQL injection, and all dynamic text printed back to the page is escaped (through `htmlspecialchars()` or the helper `e()`) to prevent cross-site scripting. Database errors are written to the server log and the visitor sees only a short, generic message.

Listing 6.6 - Database connection (config.php)

```
session_start();
// host, user, password and database name (XAMPP defaults can be overridden)
$host    = getenv('DB_HOST') ? getenv('DB_HOST') : "localhost";
$db_name = getenv('DB_NAME') ? getenv('DB_NAME') : "CakeCraft";
...
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
$conn = new mysqli($host, $db_user, $db_pass, $db_name);
```

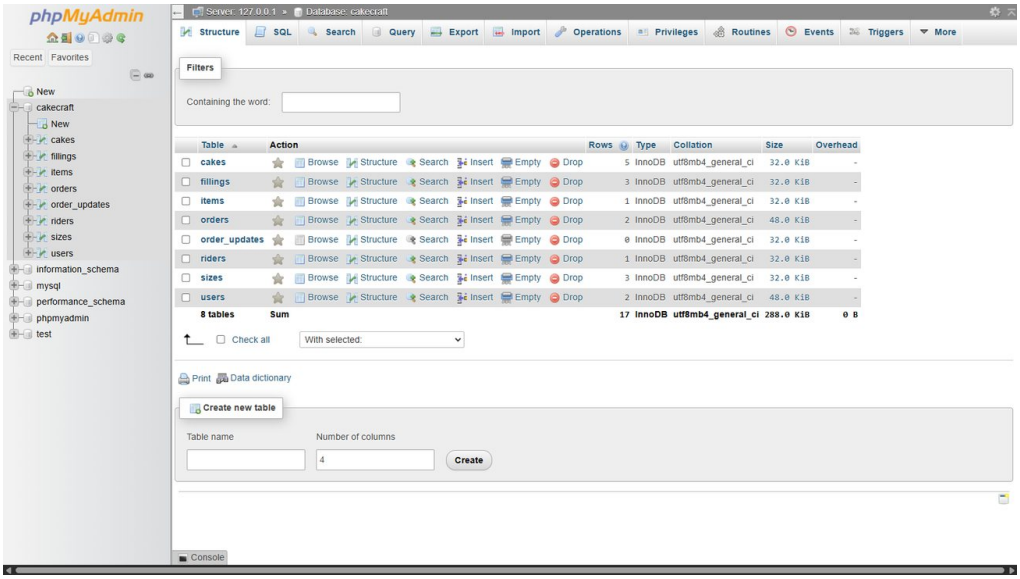


Figure 6.21: The CakeCraft database in phpMyAdmin (core tables; admins and login_attempts were added by the later security update)

6.5 CRUD Operations

The system performs all four basic database operations. The table below shows where each one is used.

Operation	Where it is used	Example
Create (INSERT)	signup.php, customize.php, admin.php, rider.php, login pages	New user, new order, new ingredient, new rider, new cake or filling, new rider update, new failed-login record
Read (SELECT)	All pages	Login lookup, menu lists with prices, pending orders, customer order history, rider assignments, stock list
Update (UPDATE)	admin.php, rider.php	Top up stock quantity, accept or decline an order, assign a rider, change a price, change the admin profile and password, rider accept or decline, mark delivered
Delete (DELETE)	admin.php, login pages	Remove a cake flavor or a filling from the menu (past orders keep their saved details); clear old failed-login records

7. Testing and Results

The system was tested manually on a local XAMPP server by following the complete life of an order with three different accounts: a customer, the admin and two riders. The main test cases, the expected results and the results actually observed are given below; the figure numbers refer to the screenshots in Section 6.

No.	Test case	Expected result	Actual result	Status
1	Register a new customer with valid details	Account is created and the dashboard opens	Dashboard opened with the welcome message (Fig. 6.2, 6.5)	Pass
2	Sign in with ID name and password	User is logged in	Customer was logged in and reached the dashboard (Fig. 6.6)	Pass
3	Choose Chocolate Truffle, Vanilla filling, Medium size	Total = 800 + 100 + 600 = 1500 TK	Total box showed 1500 TK (Fig. 6.8)	Pass
4	Place the order	Success message; order saved as pending	Message shown and order listed as Pending (Fig. 6.9, 6.10)	Pass
5	Order Red Velvet, Chocolate filling, Large size	Saved total = 900 + 120 + 1000 = 2020 TK	Admin order card showed total 2020 TK (Fig. 6.11)	Pass
6	Admin login with valid credentials	Admin panel opens	Admin panel opened (Fig. 6.3, 6.11)	Pass
7	Add ingredient 'Whipped Cream', quantity 10	Item appears in stock list	Listed as '10 left' (Fig. 6.11, 6.12)	Pass
8	Create a rider account	Rider is added and appears in the rider list	Message 'Rider has been added' and rider listed (Fig. 6.11)	Pass
9	Admin accepts the pending order	Customer sees status Accepted	Order showed Accepted (Fig. 6.19)	Pass
10	Assign the accepted order to a rider	Order is assigned; waiting for the rider	Message 'assigned ... Waiting for the rider to accept' (Fig. 6.12)	Pass

No.	Test case	Expected result	Actual result	Status
11	Rider logs in	Rider sees the new assignment	Assignment shown under New Assignments (Fig. 6.4, 6.17)	Pass
12	Rider accepts the assignment	Order moves to Ongoing Orders	Message 'You accepted order #1' and update box shown (Fig. 6.18)	Pass
13	Rider sends an update and presses Delivery Done	Updates saved; order status becomes Delivered	Customer saw 'Delivered' and the rider updates with time (Fig. 6.20)	Pass
14	Admin adds the cake 'Rainbow Cake'	Cake is added to the menu at 0 TK	Listed in the price list as 0 TK (Fig. 6.13, 6.14)	Pass
15	Admin sets the price of 'Rainbow Cake' to 600 TK	Price is updated	Message 'Price of Rainbow Cake is now 600 TK' (Fig. 6.15)	Pass
16	Admin changes the admin profile and password in Edit Credentials	Credentials are saved; the new profile and password are used at the next login	Message 'Credentials saved' shown after the new profile and password were entered twice (Fig. 6.16)	Pass

Result. All the tested functions behaved as expected. The three roles work together correctly: an order placed by the customer travels through admin approval, rider assignment, rider acceptance and delivery updates, and the customer sees the final status and every update. The prices shown in the browser matched the totals stored with the orders, menu and price changes made by the admin were reflected to customers immediately, and the admin could replace the login credentials from inside the panel. The security measures added in the final update (CSRF tokens, login-attempt limiting and the 30-minute idle logout) run in the background of these same pages.

8. Conclusion

CakeCraft successfully meets the goal of the project: it replaces informal phone-and-message ordering with a structured platform for custom cakes. Customers can sign up, design a cake, see the price instantly and follow the order to their door; the shop owner can run stock, menu, prices, orders and riders from one panel and manage the admin login credentials; and riders can accept deliveries and post live updates. The final update made the system safer by moving the admin account into the database and adding CSRF tokens, login-attempt limiting and automatic idle logout. The project allowed me to apply PHP, MySQL, HTML and CSS together in a complete application and to practise relational database design, session-based authentication, password hashing, prepared statements, CSRF protection and a multi-role workflow.

The full source code, the database script and the image assets are available on my GitHub page: https://github.com/Md-Manam-Khan/CakeCraft

Possible Future Improvements

- **Further security:** run the system over HTTPS when it is hosted online, add password reset by email, two-step verification for the admin and a log of admin actions.
- **Smarter inventory:** link ingredient stock with cake recipes so that quantities are reduced automatically when an order is accepted, with low-stock warnings.
- **Better order data:** save flavor, filling and size as separate fields with price snapshots, and keep a history of every status change, to support reports and statistics.
- **Menu management:** allow sizes to be added and removed from the admin panel.

- **Richer cake design:** a visual cake preview with frosting, toppings and message, and photographs of the cakes in the menu.
- **Notifications and payment:** email or SMS messages when the status changes and an online payment option with verification.
- **Chef dashboard and analytics:** a separate dashboard for chefs and an admin dashboard with orders, revenue and popular flavors.
- **Mobile application:** a Flutter-based mobile app built on top of the same back end, and eventually launching CakeCraft as a real cake business.

9. References

[1] CakeCraft - project source code and assets, GitHub repository: https://github.com/Md-Manam-Khan/CakeCraft

[2] The PHP Group, PHP Manual (MySQLi extension, sessions, `password_hash()` and `password_verify()`), https://www.php.net/manual/

[3] Oracle Corporation, MySQL Reference Manual, https://dev.mysql.com/doc/

[4] MDN Web Docs, HTML and CSS documentation, Mozilla, https://developer.mozilla.org/

[5] Apache Friends, XAMPP Apache + MariaDB + PHP + Perl, https://www.apachefriends.org/

[6] phpMyAdmin Project, phpMyAdmin Documentation, https://docs.phpmyadmin.net/

[7] GitHub Docs, Git and GitHub documentation, https://docs.github.com/

[8] OWASP Foundation, OWASP Cheat Sheet Series (SQL injection prevention, password storage, CSRF prevention, session management), https://cheatsheetseries.owasp.org/

[9] OpenAI, ChatGPT, used to generate the background images, https://chatgpt.com/

[10] Anthropic, Claude, used to find security loopholes and bugs in my code, https://claude.ai/